

软件过程管理第一次课后作业

基于 memogram 项目的软件过程定义与剪裁说明

姓名：许泽锋 学号：2021141510067

课程：软件过程管理

1 项目概述

本文选择本人参与开发的 `memogram` 项目作为软件过程定义对象。该项目是一个使用 Go 语言实现的 Telegram 机器人，用于将用户发送的文字、图片、文件、语音和视频保存到个人 Memos 服务中。用户通过 Telegram 命令关联自己的 Memos 访问令牌，之后可以保存消息、搜索已有备忘录，并通过按钮修改可见性、置顶状态或删除备忘录。

该系统需要与 Telegram Bot API 和 Memos API 交互，属于规模较小但具备真实外部依赖的网络应用。项目采用分层结构：Telegram 模块负责消息处理，应用模块负责业务用例，Memos 客户端负责后端接口访问，存储模块负责本地令牌持久化，配置模块负责环境变量解析。项目还包含单元测试、Dockerfile、构建脚本和 GitHub Actions 工作流。

2 软件过程定义的来源

本项目的软件过程以课程中介绍的 ISO/IEC 12207、ISO/IEC 15504 和 ISO/IEC 33001 为主要参考，并结合个人开源项目的实际情况进行剪裁。

- 1.ISO/IEC 12207:** 该标准提供软件生命周期过程框架。本文主要选取需求分析、架构设计、实现、集成、验证、确认、配置管理、问题解决和维护等活动，用于组织项目从需求到发布的工作。
- 2.ISO/IEC 15504:** 该标准强调对软件过程能力进行评估。本文不进行正式认证，而是借鉴其思想，使用可观察的工作产出判断过程是否得到执行，例如测试是否通过、代码是否经过检查、版本是否可追溯。
- 3.ISO/IEC 33001:** 该标准延续并发展了过程评估的概念和术语。本文借鉴“过程目的、过程结果、证据”的表达方式，为每个阶段定义目标、活动和产出物，使

过程可执行、可检查。

除标准外，过程定义还参考了项目已经采用的工程实践，包括 Git 版本控制、拉取请求、自动化测试、Docker 镜像构建和版本标签发布。标准提供完整框架，项目实践则决定哪些活动需要保留以及保留到什么程度。

3 适用于 memogram 的软件过程

3.1 过程模型

memogram 采用轻量级迭代式过程。每次迭代围绕一个明确需求或问题展开，完成“需求记录、影响分析、实现、验证、合并、发布或维护记录”的闭环。较小修改可以在一次提交中完成；涉及新功能、数据存储或外部 API 的修改，需要增加设计和回归验证。

需求或问题 → 分析与设计 → 实现 → 验证 → 合并 → 发布与维护

这种模型适合个人项目：开发者人数少，沟通成本低，但外部接口、访问令牌和消息数据仍要求过程具有基本的可追溯性与质量控制。

3.2 过程活动与产出物

阶段	主要活动	产出物或检查证据
需求管理	记录用户希望解决的问题，明确使用场景、输入输出和限制条件；区分新增功能、缺陷修复、依赖升级和文档修改；判断是否涉及访问令牌、用户权限或外部接口。	需求描述、Issue 或提交说明；必要时补充验收条件。
分析与设计	识别修改影响的模块，确认 Telegram、业务层、存储层和 Memos 客户端之间的职责边界；对新增命令、数据格式和配置项进行设计；优先保持已有接口稳定。	简短设计说明、接口变更说明、受影响模块清单。

阶段	主要活动	产出物或检查证据
实现	在独立分支中完成代码修改；遵守 Go 格式规范；将业务逻辑放入 <code>internal/app</code> ，将外部交互隔离在适配模块中；避免在日志中泄露令牌。	Git 提交、格式化后的源代码、必要的配置样例和 README 修改。
验证与确认	为核心逻辑补充或更新单元测试；执行 <code>gofmt</code> 、 <code>go vet</code> <code>./...</code> 和 <code>go test ./...</code> ；对机器人关键路径进行人工检查，例如关联账号、保存消息、搜索和按钮操作。	自动化测试结果、静态检查结果、人工验收记录。
集成	通过拉取请求合并代码；检查变更范围、测试结果和兼容性；确认依赖修改同步写入 <code>go.mod</code> 和 <code>go.sum</code> 。	拉取请求、评审记录、通过的 CI 工作流。
发布	为可发布版本创建版本标签；由工作流再次运行测试，构建多平台二进制文件和 Docker 镜像，并生成校验和。	版本标签、构建产物、Docker 镜像、发布说明和校验和文件。
维护	收集运行问题；根据严重程度安排修复；定期升级依赖并执行回归测试；对安全相关问题优先处理。	缺陷记录、修复提交、依赖升级提交、回归测试结果。
配置管理	使用 Git 管理源码和文档；使用环境变量管理服务地址、机器人令牌和允许用户列表；只提交 <code>.env.example</code> ，不提交真实密钥。	版本历史、配置样例、 <code>.gitignore</code> 、版本标签。

3.3 质量检查点

为了使过程定义能够真正执行，而不是停留在文档层面，设置以下检查点：

- 1.合并前必须确认代码已格式化，并执行 `go vet ./...` 和 `go test ./...`。
- 2.修改令牌存储、权限控制或外部 API 访问逻辑时，必须增加针对异常情况的测试。
- 3.修改环境变量、命令用法或部署方式时，必须同步更新配置样例或 README。

- 4.发布前必须重新运行测试；构建失败或测试失败时不得发布。
- 5.真实的 Telegram Bot Token 和 Memos Access Token 不得进入 Git 仓库、日志或公开 Issue。

4 剪裁思路

ISO/IEC 12207 面向不同规模的软件项目，包含较完整的生命周期过程。若直接应用于个人项目，会产生明显高于开发本身的管理成本。因此，本项目按照“保留必要控制、降低形式成本”的原则进行剪裁。

4.1 保留的过程

需求、实现、验证、配置管理、问题解决和维护是本项目的核心过程，必须保留。虽然项目规模不大，但它处理用户令牌并依赖两个外部服务。一旦出现错误，可能造成消息无法保存、令牌泄露或兼容性问题。因此，自动化测试、版本控制和发布前检查不能省略。

设计活动也予以保留，但采用轻量方式执行。对于一般修改，只需在 Issue 或拉取请求中说明影响模块；对于存储格式、权限策略和外部接口变更，则需要更明确的设计说明。这样既能控制风险，也不会为简单修改增加过重负担。

4.2 简化或删除的过程

本项目不涉及甲乙双方合同和采购，因此不单独定义获取过程、供应过程和合同管理过程。项目目前以个人维护为主，也不设立专门的项目管理办公室、质量保证部门和多级审批流程。进度管理不使用复杂的工作量模型，而是按功能和问题拆分短迭代。

文档过程也进行了简化。项目保留 README、配置样例、提交历史、拉取请求说明和发布记录，不为每次修改编写完整的需求规格说明书、概要设计说明书和详细设计说明书。对于小型开源项目，这些轻量产出物已经能够支持追溯和维护。

4.3 过程能力的轻量评估

参考 ISO/IEC 15504 和 ISO/IEC 33001 的过程评估思想，可以使用下列问题定期检查过程执行情况：

- 1.每次修改是否能够追溯到明确需求、缺陷或维护原因？
- 2.核心逻辑是否有测试，合并前 CI 是否通过？
- 3.发布版本是否对应标签，构建产物是否能够复现？

- 4.配置样例和 README 是否与当前代码一致？
- 5.是否出现密钥泄露、未经验证直接发布等问题？

若这些问题能够持续得到肯定回答，说明过程已基本建立并受到管理。若项目以后发展为多人协作或面向更多用户，则应增加正式的 Issue 模板、评审责任人、安全扫描、变更日志和发布回滚方案。

5 总结

本文依据 ISO/IEC 12207 的生命周期框架，并借鉴 ISO/IEC 15504 和 ISO/IEC 33001 的过程评估思想，为 memogram 定义了轻量级迭代软件过程。该过程覆盖需求、分析设计、实现、验证、集成、发布、维护和配置管理，同时根据个人开源项目的特点剪裁了合同管理、复杂组织管理和重型文档。剪裁后的过程保留了必要质量控制，并能利用 Git、自动化测试和 CI 工作流形成可执行、可追溯的开发闭环。

参考资料

- 1.ISO/IEC 12207，系统与软件工程：软件生命周期过程。
- 2.ISO/IEC 15504，信息技术：过程评估。
- 3.ISO/IEC 33001，信息技术：过程评估：概念和术语。
- 4.memogram 项目 README、源代码、测试文件和 GitHub Actions 工作流。